Git vs GitHub



DEVELOPER FOUNDATIONS

Five essential differences every developer should understand before collaborating on code.

Prepared by:

- Zahra Haratian
- Nika Farzani



With Sincere Appreciation to:

Professor Mohammad Saleh

University of Science and Culture

First question you may have

Why were Git and GitHub created?

Git was created to track changes in code and help developers work on projects more easily.

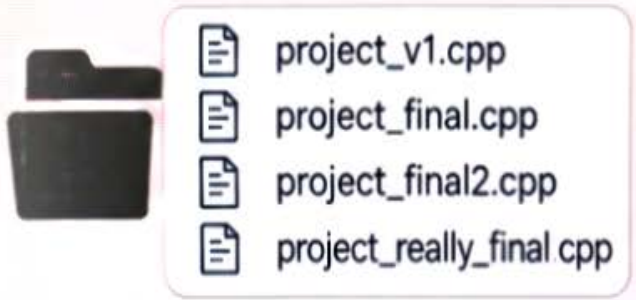
GitHub was created to help developers store, share, and collaborate on code online.



The problem (without Git/GitHub)

When working on a project, developers face several issues:

- **No history:**
- **Confusing file names:**
- **Hard to collaborate:**
- **No central place:**

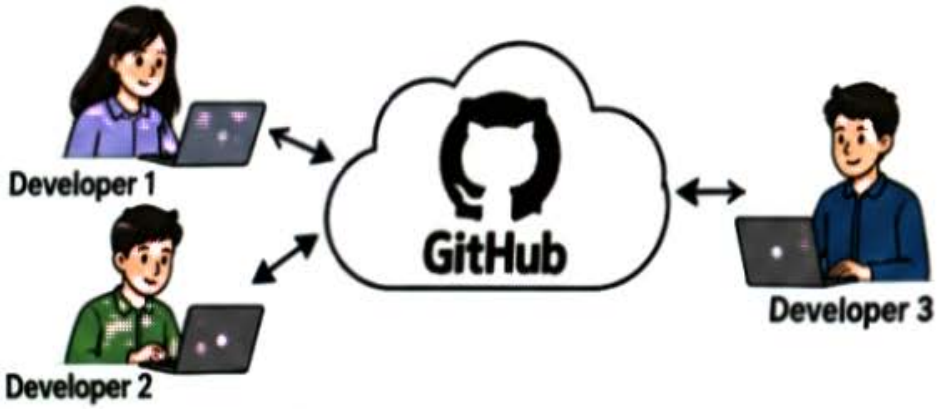




The solution (with Git + GitHub)

Git and GitHub were created to solve these problems:

- **Track changes:**
- **Keep everything organized:**
- **Work together easily:**
- **Have a central place:**



Example

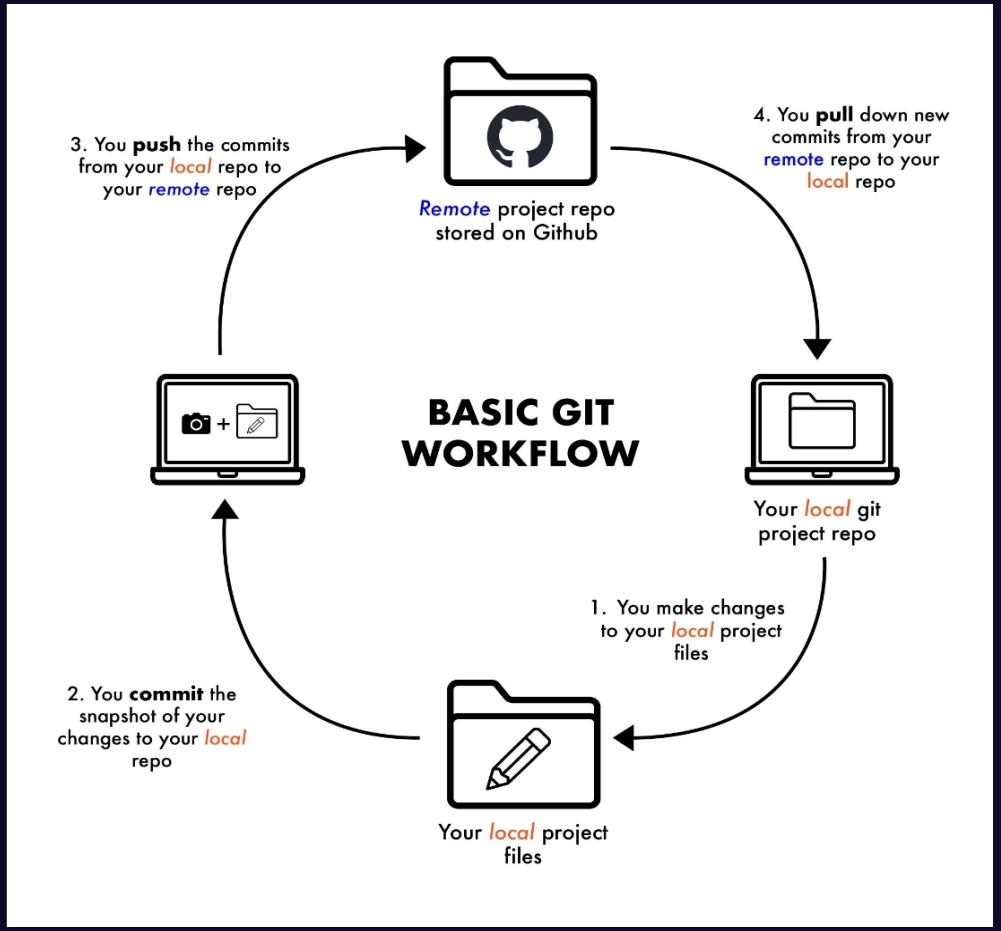
Imagine three students working on the same project.

Without Git, they may lose their code or get confused about different versions.

Git helps them save changes and return to previous versions.

GitHub allows them to share their code and work together online.

In short, Git manages code changes, and GitHub makes collaboration easier.



DIFFERENCE 01 · PURPOSE

Version control vs collaboration platform

Git protects the history of a codebase. GitHub organizes the human workflow around that history.

GIT ANSWERS

What changed?

When did it change, who changed it, and how can I restore an earlier version?

GITHUB ANSWERS

How does the team respond?

How can people discuss, review, approve, track, and share those changes?

DIFFERENCE 02 · WHERE THEY RUN

Local software vs hosted service

Git runs locally

It is installed on your computer and operates on a local repository. Many everyday actions work offline.

Create commits, inspect history, and create branches before connecting to a remote host.

GitHub runs online

It provides hosted repositories, accounts, organizations, permissions, and a web interface.

Online collaboration requires access to the hosted service.

Practical effect: Git is available anywhere your files and Git installation are available. GitHub is the shared online meeting place.

DIFFERENCE 03 · SCOPE

One project history vs a shared project space

The repository is the common object, but each tool gives it a different boundary.

Git repository

The project plus its complete version history, distributed to each contributor in a distributed workflow.

GitHub repository

A shared online home with visibility, access controls, issues, pull requests, discussions, and project context.

The boundary

Git manages the repository and its history. GitHub adds the people, permissions, and conversations around it.

DIFFERENCE 04 · INTERACTION MODEL

Commands vs review workflow

GIT COMMANDS

`git status`

`git add` · `git commit`

`git branch` · `git merge`

`git log`

Local actions that inspect, record, organize, and combine project history.

GITHUB WORKFLOWS

Open a pull request.

Comment on a diff and request changes.

Approve, merge, or track an issue.

Pull requests propose, discuss, review, and merge changes. Git itself has no pull-request website or review inbox.

Core history vs an extensible ecosystem

Git supplies the version-control engine. GitHub surrounds it with services for teams and delivery.



Git core

Track snapshots, branches, merges, remotes, and authorship in a repository.



GitHub layer

Code review, issue tracking, organizations and teams, notifications, security features, and GitHub Actions automation.

The boundary matters

Git can be paired with other hosting services. GitHub is one specific platform with its own integrated ecosystem.

How they work together in a normal contribution

01

Start local

Clone or initialize a repository with Git.

02

Make a branch

Edit files without disturbing the main line.

03

Commit

Record the change with Git.

04

Push

Send the branch to GitHub.

05

Open a PR

Invite discussion and review.

06

Merge

Run checks, respond to feedback, and merge the approved change.

Think local history first, shared review second.

Six Technical Terms in Git and GitHub

1. Repository (Repo): A place where project files and their history are stored.
2. Commit: A saved record of changes made to a project.
3. Pull Request (PR): A request to review and merge changes into a project.
4. Merge: Combine changes from different branches.
5. Pull: Get the latest changes from a remote repository.
6. Push: Send your changes to a remote repository.

Final questions

1st: What's the difference between `git add` and `git commit`?

-- `git add` stages files so they're ready to be recorded, and `git commit` saves those staged changes as a version in the project history.

2nd: What is a Pull Request?

-- It's a request to review and merge the changes from one branch into the main branch. Before the merge, others can comment on the changes or approve them.

References :

1. ChatGPT: https://chatgpt.com/

2.Youtube: https://youtu.be/Ez8F0nW6S-w?si=LK7fZSgP101o6YAP

3.Websites:
https://www.w3schools.com/git/git_intro.asp
https://docs.github.com/en
https://www.datacamp.com/blog/git-vs-github


def main():
print("Hello, World!")
if __name__ == "__main__":
main()
Better Code Better Tomorrow
Learn Build Grow
Computer Science
Programming
Technology
Thank you for your attention!
We hope you found the information useful.